



Universidade de São Paulo - São Carlos
Instituto de Ciências Matemáticas e de Computação

SCC-541– Laboratório de Bases de Dados

Trabalho Prático T4 – Exercícios sobre Visões (*views*)

Docente: Caetano Traina Jr. — caetano@icmc.usp.br

PAE: Gabriel Lucca de Melo — melogabriel@usp.br 1º Semestre de 2026

Data para entrega: 20 de maio de 2026

Datasets

Base de dados da Fórmula 1 - FIA, com cidades e aeroportos do mundo.

A base de dados a ser utilizada é a mesma estruturada ao longo das atividades da disciplina. Os arquivos necessários para a carga de dados estão disponíveis na **Seção 3 do Moodle (e-Disciplinas)**, podendo ser acessados no link a seguir: [link de acesso](#).

Os dados disponibilizados foram ligeiramente modificados para facilitar o desenvolvimento da atividade. No entanto, caso desejem, vocês podem utilizar os dados originais, por exemplo, para incluir informações atualizadas das últimas corridas.

Nesse caso, os dados podem ser obtidos nos seguintes *sites*:

- Dados da Fórmula-1: [Formula 1 Championships \(1950-2025\) – Kaggle](#)
 - Países e Cidades do planeta: [GeoNames](#)
 - Aeroportos: [OurAirports](#)
-

Atividades da semana

- O objetivo deste trabalho é praticar o uso de **visões** (*views*) nos SGBDs.
 - É obrigatória a criação de *views* em todos os exercício, exceto se a questão apontar o contrário.
-

Exercícios

Exercício 1) Crie uma visão para manter os aeroportos vinculados a cidades brasileiras.

Devem ser incluídos os dados do aeroporto ([name](#), [latitude_deg](#), [longitude_deg](#)), os dados do país ([name](#) e continente) e os dados da cidade vinculada ([name](#) e [population](#)).

Para fazer as junções, considere a nova modelagem da base:

- o atributo [city_id](#) de [airports](#) se refere a [id](#) em [cities](#);

- o atributo `country_id` de `cities` se refere a `id` em `countries`;
 - o atributo `continent_id` de `countries` se refere a `id` em `continents`.
- (a) Crie a visão materializada, que deve ser chamada `Aeroportos_Brasileiros`, e mostre no relatório um exemplo com algumas tuplas mantidas e o número de tuplas dessa visão.
 - (b) Discuta as vantagens e desvantagens de se utilizar visões materializadas, medindo tempos médios de execução (use pelo menos 5 execuções) e consumo de armazenamento obtidos pela execução de consultas que usem a visão materializada e a consulta equivalente sobre as tabelas-base.
 - (c) Após criar a visão materializada, execute ao menos uma atualização nas tabelas-base que possa afetar seu resultado e mostre no relatório a necessidade do comando `REFRESH MATERIALIZED VIEW`.

Exercício 2) Com base na questão anterior, elabore uma segunda visão que apresente apenas os aeroportos que não estejam vinculados a nenhuma cidade. Nomeie essa visão como `Aeroportos_sem_cidades`.

Dica: neste caso, um aeroporto sem cidade é aquele cujo atributo `city_id` está nulo.

Depois, crie uma terceira visão que contenha apenas as cidades brasileiras com população de no mínimo 100 mil habitantes. Essa visão deve ser chamada `Cidades_brasileiras`.

Em seguida, associe a cada aeroporto de `Aeroportos_sem_cidades` as cidades brasileiras da visão `Cidades_brasileiras` que estejam a no máximo 10Km. Apresente o nome do aeroporto, o nome e a população da cidade, e a distância entre o aeroporto e a cidade. Note que, para cada aeroporto, podem ser encontradas várias cidades.

Atenção: como esses aeroportos não possuem cidade vinculada, eles não podem ser classificados diretamente por país a partir da modelagem atual. Nesta questão, devem ser consideradas como candidatas apenas cidades da visão `Cidades_brasileiras`.

Deve ser definida uma função para calcular as distâncias entre pontos na superfície da Terra, usando a distância de Haversine. Para isso, é possível definir uma função pela fórmula, ou pode ser usada a função `EarthDistance` já disponível nas `EXTENSIONS` `EarthDistance` e `CUBE`, tal como exemplificado a seguir:

```
CREATE EXTENSION IF NOT EXISTS Cube;
CREATE EXTENSION IF NOT EXISTS EarthDistance; - Inclui EarthDistance(),
                                                - precisa de Cube

SELECT A.name, C.name, Earth_Distance(
                        LL_to_Earth(A.latitude_deg, A.longitude_deg),
                        LL_to_Earth(C.latitude, C.longitude))
FROM airports A, cities C;
```

Alguns pontos importantes sobre esse código:

- Os comandos `CREATE EXTENSION` instalam as extensões necessárias para usar as funções de cálculo de distância.
- A função `LL_to_Earth` retorna a localização de um ponto na superfície terrestre a partir de uma Latitude (argumento 1) e Longitude (argumento 2).
- A função `EarthDistance` retorna a distância entre dois pontos na superfície terrestre, segundo a distância de Haversine.

Sugestão: se desejarem, utilizem uma visão inline (`WITH`) para isolar o cálculo da distância e evitar repetição de código.

Exercício 3) Crie uma visão chamada `Circuitos_completa` que apresente informações dos circuitos e de sua localização geográfica completa.

Devem ser apresentados os atributos:

- `name`, `lat` e `long` da tabela `circuits`;
- `name` da cidade em `cities`;
- `name` e `code` do país em `countries`;
- `name` do continente em `continents`.

Essa visão deve apresentar todos os circuitos, independentemente de estarem ou não vinculados a alguma cidade.

Considere que:

- o atributo `city_id` em `circuits` se refere a `id` em `cities`;
- o atributo `country_id` em `cities` se refere a `id` em `countries`;
- o atributo `continent_id` em `countries` se refere a `id` em `continents`.

Apresente no relatório:

- o número de tuplas dessa visão;
 - um exemplo variado de algumas das tuplas obtidas.
-

Exercício 4) Considerando a questão 2, crie uma visão chamada `Problemas_aeroportos` que apresente apenas os aeroportos sem cidade vinculada e para os quais exista ao menos uma cidade brasileira da visão `Cidades_brasileiras` localizada a, no máximo, 10Km de distância.

Devem ser apresentados os atributos:

- `id`, `name`, `latitude_deg` e `longitude_deg` da tabela `airports`;
- o nome da cidade candidata;
- a população da cidade candidata;
- a distância entre o aeroporto e a cidade candidata.

Atenção:

- um aeroporto problemático, nesta questão, é um aeroporto cujo atributo `city_id` está nulo;
- devem ser consideradas apenas cidades da visão `Cidades_brasileiras`;
- para cada aeroporto, podem existir várias cidades candidatas.

Esses casos devem ser interpretados como **casos candidatos à correção**, isto é, aeroportos para os quais a base ainda não registra cidade vinculada, mas para os quais já existe evidência geográfica de possível associação.

Dica: pode ser usada a função `Earth_Distance` juntamente com as extensões `Cube` e `EarthDistance`. Se desejarem, utilizem uma visão inline (`WITH`) para isolar o cálculo de distância.

(a) Crie a visão.

(b) Responda: por que esses aeroportos aparecem nessa visão?

No relatório, mostre exemplos de tuplas dessa visão e explique o motivo de cada caso encontrado.

Exercício 5) Crie uma visão chamada `Correcao_aeroportos` que liste apenas os aeroportos vinculados aos casos identificados na questão anterior, isto é, aeroportos sem cidade vinculada e que possuam ao menos uma cidade brasileira candidata a, no máximo, 10Km de distância.

Essa visão deve apresentar os atributos `id`, `name`, `latitude_deg`, `longitude_deg` e `city_id` da tabela `airports`. Não deve ser executada nenhuma junção com a tabela `cities` para criar essa visão.

Depois, identifique cidades candidatas para os aeroportos presentes nessa visão e atualize o atributo `city_id` dos aeroportos utilizando a visão criada, de modo que esses aeroportos passem a ficar vinculados a cidades brasileiras próximas.

Para a escolha das cidades candidatas, considere preferencialmente a menor distância entre as coordenadas do aeroporto e as coordenadas da cidade. Quando houver mais de uma cidade candidata, pode ser escolhida a cidade mais próxima.

Execute os comandos e mostre o resultado da execução, junto com o número de tuplas afetadas, para cada uma das correções realizadas.

Sugestão:

- use a visão `Problemas_aeroportos` para identificar as cidades candidatas;
- se desejar, utilize uma visão inline (`WITH`) para selecionar, para cada aeroporto, apenas a cidade mais próxima;
- após as correções, consulte novamente as visões `Aeroportos_sem_cidades` e `Problemas_aeroportos` para verificar o efeito das atualizações.

Entrega da semana

Cada equipe deverá entregar dois arquivos:

1. Um arquivo com o *script* no formato `.zip`, contendo um arquivo `.sql` com os comandos `SQL` utilizados para cada atividade. (subitens de cada exercício devem estar no mesmo arquivo `.sql`, separados adequadamente por comentários explicativos).
 2. Um arquivo com o relatório SUCINTO no formato `.pdf`.
 3. O relatório deverá apresentar testes (pelo menos um por questão) para cada uma das visões e comandos criados em cada questão, com uma captura de tela do resultado para cada teste.
 4. Quando os resultados forem muito longos, o grupo deverá apresentar somente as primeiras tuplas (por exemplo, as 10 primeiras linhas de uma tabela resultante).
-

Atenção

- O relatório deve evidenciar claramente as decisões adotadas pelo grupo e justificar as escolhas realizadas no desenvolvimento do trabalho.
- Não serão aceitos projetos feitos à mão e a organização clara das respostas também é um ponto avaliado.
- Plágio será avaliado com nota zero.
- Os arquivos devem ser submetidos:

até às **19:00** do dia **20 de maio de 2026**
com a **postagem apenas no moodle**
e-Disciplinas.

Bom Trabalho!